

8.2 Continual Learning and Forgetting

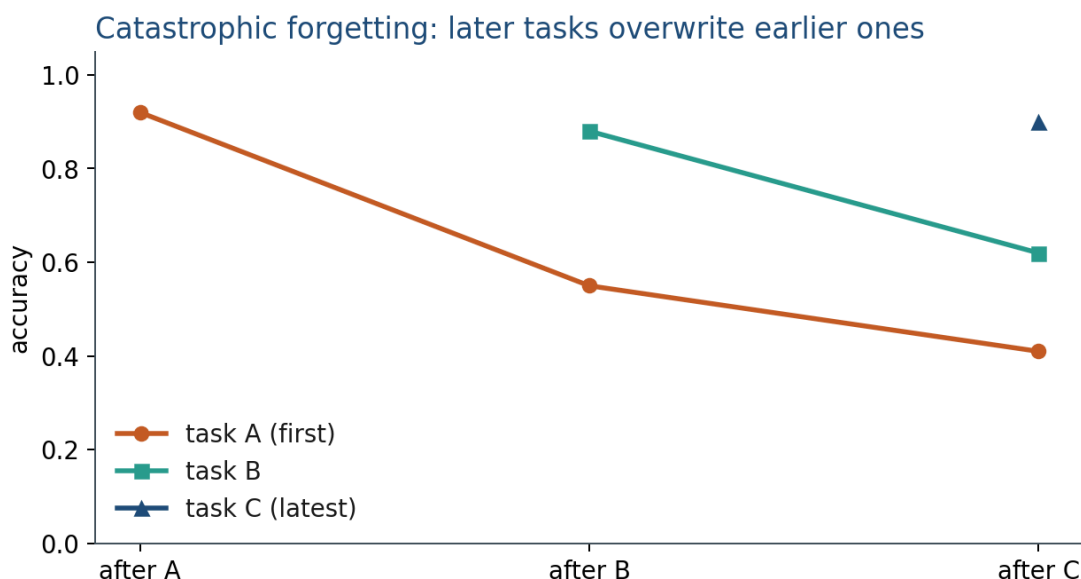
Train on task B and task A slips. Replay, freeze, or isolate the change.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Train on task B after task A and **task A's accuracy falls**. That drop is **catastrophic forgetting**. It is not a bug in one optimizer; it is what a single shared W does when the new gradient points away from the old solution.

1. Why the old skill dies

The SFT (or pretrain) minimum for A is a point in weight space. Fine-tuning on B walks away from it. In an MLP, the same hidden units implemented both functions; XOR then AND in Lab 8 is the cartoon. In an LLM, new instruction data overwrites features that still mattered for coding, or for a language you are not currently sampling.



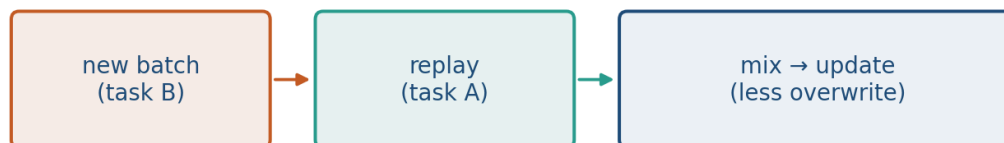
Measuring it is part of the method: keep a **held-out A set** and plot it every k steps of B. If you only report B, you did not look. Instruction SFT is a special case: the “old task” is everything the base model already did (code, languages, refusal style).

A single shared W cannot be at two distant minima at once unless the tasks agree. XOR and AND disagree on the same four bit-pairs; that is why Lab 8 is small enough to plot. In an LLM the disagreement is softer (new hospital notes vs old coding skill) but the picture is the same walk in weight space.

2. Replay and other patches

Replay: mix a buffer of A examples into the B batches so the gradient still has a component that preserves A. The buffer can be real stored rows or **pseudo-replay** (sample the old model). This is the idea you will implement in spirit: do not throw A away.

Replay (or freeze, or LoRA) is how you keep task A alive



Other knobs: smaller learning rates, freeze early layers, **regularize toward** W_A (EWC-style penalties), or attach a **task adapter** and leave W frozen (note 8.3). Adapters are often the cleaner LLM answer: you add B without moving A's weights.

A 50/50 mix of A and B in each batch is the simplest replay. If you cannot store A, generate from π_A and treat those strings as A labels (they are imperfect).

Replay still **moves** W . It only keeps a component of ∇L_A in the batch. LoRA **does not move** W . Those are different claims; do not mix them in one sentence of a report.

3. Continual setups you will actually run

Instruction collections arrive in waves (new tool, new hospital, new semester). A legal design is: freeze the base, train LoRA on B, keep the A LoRA. Merging adapters is a separate, lossy step. If you must update one set of weights, replay A on purpose.

4. Teaching this note

About **30 minutes** at the board: XOR-then-AND on a shared hidden layer, a sketch of accuracy-A vs steps-of-B, then "freeze W , train an adapter." Play LoRA with the pause on **adapter vs overwrite** (frozen W). Lab 8 section 2 is the tiny forget curve.

Leave a table on the board: method | does W move? | need A data? Replay: yes, yes. Frozen LoRA: no, no (for A). EWC: yes, no (needs a Fisher sketch). Students should be able to fill that without notes.

5. Worked example

Two tasks, one hidden layer of width 2. Train XOR to 100% on a four-point set. Then train AND on the same four (x_1, x_2) bits, same W . AND is linearly separable; XOR is not. After enough AND steps, XOR accuracy typically falls toward chance (2/4). That is forgetting as geometry: the hidden features rotated to serve AND.

Replay: each batch is 2 XOR rows + 2 AND rows. The XOR loss stays in the gradient. You may not hit 100% on both; you will not usually zero XOR.

Adapter: freeze W_{XOR} , learn a LoRA (or a second head) for AND. XOR accuracy on the frozen net **cannot** move. That is the thing replay cannot promise: replay still updates the shared weights.

Plot: x-axis = AND steps {0, 50, 100, 200}, y-axis = XOR held-out accuracy. If the line is missing from a report, the method is incomplete.

Numbers: XOR chance is $2 / 4 = 0.5$. If XOR goes $4 / 4 \rightarrow 2 / 4$ while AND goes $2 / 4 \rightarrow 4 / 4$, you traded tasks. Replay might land at $3 / 4$ and $3 / 4$. LoRA-for-AND keeps XOR at $4 / 4$ on the frozen net.

6. Where students get stuck

- Reporting only task-B accuracy after a sequential train.
 - Calling any accuracy drop “overfitting.” Overfitting is train vs test on **one** task; forgetting is A vs B.
 - Mixing replay with LoRA in the writeup. Say which weights moved.
-

7. Video

[Umar Jamil — LoRA explained](#). Play the opening until W is frozen and only A, B train (first ~10–15 min). Pause on **adapter vs overwrite**: that is this note’s fix. Full LoRA algebra is the next note.

8. Practice

1. You fine-tune only on AND after XOR. What should happen to XOR accuracy if the hidden layer is tiny and shared?
2. Why is sampling the *old* model a form of replay even when you cannot store the old dataset?
3. Name one reason LoRA (next note) can avoid forgetting that replay cannot: the base W never moves.
4. Task A had 200 held-out items, 180 correct (90%). After B, 120 correct. What is the forgetting gap in points, and what must you still check on B?
5. Batches of 8. Replay mixes 2 A examples with 6 B. What fraction of the gradient’s data is still A, and why might XOR still die if that fraction is this small?